



Zewail City for Science and
Technology

2025-2026

<i>Course:</i>	Numerical Analysis
<i>Code:</i>	MATH 307
<i>type</i>	Final Report

<i><u>Due Date:</u></i>	May 17, 2026
<i><u>Project name:</u></i>	Bilinear Interpolation for Image Rescaling

Under The Supervision of

Dr. Ahmed Abdelsamea

by:

<i>Name</i>	<i>ID</i>	<i>Email</i>
Mostafa Hisham	202201998	Mostafa Hisham 202201998
Nour-Eldin Mohamed Saber	202201310	Nour Eldin 202201310

Table of contents:

1. Abstract.....	4
2. Introduction.....	4
3. Problem Definition & Mathematical Formulation.....	5
3.1 Formal Setup and Variable Definitions.....	5
3.2 Step-by-Step Derivation: Sequential Lagrange Interpolation.....	5
Stage 1 — Interpolation Along the x-Direction (y Held Fixed).....	5
Stage 2 — Interpolation Along the y-Direction.....	6
Full Expansion and Compact Area-Weighted Form.....	6
3.3 Worked Numerical Example (from Course Lecture Notes).....	6
4. Methodology & Software Architecture.....	7
4.1 Context: From 1D Splines to 2D Bilinear Interpolation.....	7
4.2 Numerical Algorithm: Inverse Mapping, Clamping, and Weighting.....	7
4.3 Software Architecture: Microservices Deployment.....	7
5. Comparison of Interpolation Techniques.....	8
5.1 Nearest-Neighbor Interpolation (Baseline).....	8
5.2 Bilinear Interpolation (This Project).....	9
5.3 Higher-Order Methods: Bicubic Interpolation and Beyond.....	9
5.4 Analytical Summary Comparison Table.....	9
6. Results & Analysis.....	10
6.1 Test Case 1: Synthetic $2 \times 2 \rightarrow 8 \times 8$ Upscaling (Mathematical Validation).....	10
Nearest-Neighbor Result.....	10
Bilinear Interpolation Result.....	10
6.2 Test Case 2: Real-World $512 \times 512 \rightarrow 2048 \times 2048$ Image Upscaling.....	11
Nearest-Neighbor Result.....	11
Bilinear Interpolation Result.....	11
Quantitative Observations.....	11
7. Conclusion.....	12
8. Appendances.....	12
9. Work Division.....	13
The project tasks were divided between the team members as follows:.....	13
10. Results.....	13
11. References.....	14

1. Abstract

Digital images are inherently discrete grids of pixels, and geometric operations such as **image rescaling** inevitably produce non-integer (sub-pixel) coordinates that cannot be resolved by simple table look-up. This project applies the principles of **bilinear interpolation**—a two-dimensional extension of Lagrange linear interpolation studied in MATH 307—to reconstruct missing pixel intensities in a principled, mathematically rigorous manner. The core formula decomposes the 2D estimation problem into two sequential 1D interpolations, yielding a compact area-weighted average of the four nearest-neighbor pixels surrounding every sub-pixel target location.

The project delivers both a thorough mathematical treatment and a production-grade software system. The backend is implemented as a **FastAPI microservice** powered by NumPy matrix operations, the frontend is a **Streamlit** interactive web application, and the complete stack is containerized with Docker and deployed publicly on Hugging Face Spaces. Validation is performed on two test cases: a controlled $2 \times 2 \rightarrow 8 \times 8$ synthetic upscaling (proving mathematical correctness through known color blending) and a real-world $512 \times 512 \rightarrow 2048 \times 2048$ photograph upscaling, where bilinear interpolation demonstrably outperforms the nearest-neighbor baseline by producing smooth, gradient-continuous facial features. The results confirm that classical numerical analysis methods translate directly into practical, deployable image-processing solutions.

2. Introduction

A digital image is a finite, two-dimensional array of picture elements—pixels—each carrying an integer intensity value at an integer coordinate location on a regular grid. This discrete representation is perfectly suited to digital storage and display hardware; however, it introduces a fundamental challenge when the image must undergo **geometric transformations** such as scaling, rotation, or shearing (Gonzalez & Woods, 2018).

Image rescaling is one of the most frequently performed operations in digital image processing, appearing everywhere from smartphone cameras to medical imaging pipelines. When an image is enlarged or reduced by an arbitrary factor, each pixel in the destination (output) image must be assigned an intensity derived from the source (input) image. The standard approach—known as **inverse affine mapping**—works backwards: for every output pixel at integer position (u, v) , a transformation is applied to find the corresponding source location (x_i, y_i) . Because the scaling factor is generally irrational relative to the pixel grid, the result is a non-integer, sub-pixel coordinate. For example, enlarging a 100×100 image to 200×200 using a scale factor of 2 maps the output pixel $(1, 0)$ back to the fractional source location $(0.5, 0.0)$, which lies precisely between two source pixels. No pixel in the original image carries a value at that location.

This is where **numerical interpolation** becomes indispensable. As Chapra & Canale (2021) note, interpolation methods for one-dimensional problems can be systematically extended to multidimensional cases through separable application of the same linear schemes. Among available strategies, **bilinear interpolation**—which combines the four nearest known pixels in a distance-weighted average—stands out as the standard numerical scheme for sub-pixel intensity estimation in 2D image processing (Sayed AbdelSamea, 2026). Unlike the simpler nearest-neighbor method, which assigns every output pixel the single closest source pixel and produces blocky, pixelated artifacts, bilinear interpolation computes a smooth, continuous approximation of the underlying image signal. Ullah et al. (2025) confirm through comparative analysis that bilinear interpolation consistently outperforms nearest-neighbor approaches in visual quality across a wide range of upscaling dimensions.

This report presents a complete account of the mathematical derivation, software implementation, comparative analysis, and experimental results of a bilinear image rescaling system developed for MATH 307. Section 3 provides the formal problem definition and mathematical formulation. Section 4 details the computational methodology and software architecture. Section 5 compares bilinear interpolation against competing methods. Section 6 presents and analyses the experimental results. Section 7 draws conclusions and identifies future directions.

3. Problem Definition & Mathematical Formulation

3.1 Formal Setup and Variable Definitions

Consider a source image defined on a regular integer grid. An inverse affine mapping—with horizontal scale factor s_x and vertical scale factor s_y —maps each output pixel (u, v) to the fractional source coordinates:

$$x_i = u / s_x, \quad y_i = v / s_y$$

Because (x_i, y_i) is generally non-integer, we identify the four surrounding integer-grid neighbors using floor operations. The complete set of variables is defined as follows:

Variable	Definition
x_i, y_i	Fractional (sub-pixel) spatial coordinates of the target point in the source image
x_1, x_2	Left and right bounding integer column coordinates: $x_1 = \lfloor x_i \rfloor, x_2 = x_1 + 1$
y_1, y_2	Top and bottom bounding integer row coordinates: $y_1 = \lfloor y_i \rfloor, y_2 = y_1 + 1$
dx	Horizontal fractional distance: $dx = x_i - x_1, dx \in [0, 1)$
dy	Vertical fractional distance: $dy = y_i - y_1, dy \in [0, 1)$
$f(x_1, y_1)$	Pixel intensity at the top-left corner of the enclosing cell
$f(x_2, y_1)$	Pixel intensity at the top-right corner of the enclosing cell
$f(x_1, y_2)$	Pixel intensity at the bottom-left corner of the enclosing cell
$f(x_2, y_2)$	Pixel intensity at the bottom-right corner of the enclosing cell
$f(x_i, y_i)$	Unknown interpolated pixel intensity — the quantity to be estimated

3.2 Step-by-Step Derivation: Sequential Lagrange Interpolation

Bilinear interpolation proceeds by applying the standard **1D Lagrange linear interpolation** formula twice—first across the x-direction, then across the y-direction—exploiting the separability of the rectangular pixel grid (Sayed AbdelSamea, 2026; Chapra & Canale, 2021).

Stage 1 — Interpolation Along the x-Direction (y Held Fixed)

For a query point at column x_i , we interpolate between the left and right columns at each of the two bounding rows, using the Lagrange basis form:

Top row ($y = y_1$):

$$f(x_i, y_1) = [(x_i - x_2) / (x_1 - x_2)] \cdot f(x_1, y_1) + [(x_i - x_1) / (x_2 - x_1)] \cdot f(x_2, y_1)$$

Bottom row ($y = y_2$):

$$f(x_i, y_2) = [(x_i - x_2) / (x_1 - x_2)] \cdot f(x_1, y_2) + [(x_i - x_1) / (x_2 - x_1)] \cdot f(x_2, y_2)$$

In each expression, the Lagrange basis weights sum to 1, ensuring the interpolation passes exactly through both known values at x_1 and x_2 .

Stage 2 — Interpolation Along the y-Direction

Having produced two intermediate values at (x_i, y_1) and (x_i, y_2) , we apply the Lagrange formula once more in the y-direction:

$$f(x_i, y_i) = [(y_i - y_2) / (y_1 - y_2)] \cdot f(x_i, y_1) + [(y_i - y_1) / (y_2 - y_1)] \cdot f(x_i, y_2)$$

Full Expansion and Compact Area-Weighted Form

Substituting the two Stage-1 expressions into Stage 2 and simplifying—exploiting that $x_2 = x_1 + 1$ and $y_2 = y_1 + 1$ so that all denominators reduce to 1 in magnitude—yields the fully expanded bilinear expression (Sayed AbdelSamea, 2026):

$$f(x_i, y_i) = [(x_i - x_2)(y_i - y_2) / (x_1 - x_2)(y_1 - y_2)] \cdot f(x_1, y_1) + [(x_i - x_1)(y_i - y_2) / (x_2 - x_1)(y_1 - y_2)] \cdot f(x_2, y_1) \\ + [(x_i - x_2)(y_i - y_1) / (x_1 - x_2)(y_2 - y_1)] \cdot f(x_1, y_2) + [(x_i - x_1)(y_i - y_1) / (x_2 - x_1)(y_2 - y_1)] \cdot f(x_2, y_2)$$

Substituting $dx = x_i - x_1$ and $dy = y_i - y_1$ (each in $[0, 1)$) simplifies every Lagrange coefficient and gives the elegant, compact area-weighted formula:

$$f(x_i, y_i) = (1-dx)(1-dy)f(x_1, y_1) + dx(1-dy)f(x_2, y_1) + (1-dx)dyf(x_1, y_2) + dx \cdot dy \cdot f(x_2, y_2)$$

This expression reveals that the interpolated value is a **convex combination** of the four neighboring pixels, weighted by the rectangular areas that the sub-pixel point creates when it partitions its enclosing 1×1 pixel cell. This geometric interpretation—the area opposite a corner weights the corner's contribution—is precisely why the method is called *bilinear*: the interpolating surface is linear in each coordinate direction separately.

3.3 Worked Numerical Example (from Course Lecture Notes)

As demonstrated in Sayed AbdelSamea (2026), consider a heated rectangular plate with measured temperatures at four corner positions:

Corner	Coordinate (x, y)	Temperature T
Top-left	(2, 1)	T = 60 °C
Top-right	(9, 1)	T = 57.5 °C
Bottom-left	(2, 6)	T = 55 °C
Bottom-right	(9, 6)	T = 70 °C

To estimate the temperature at the interior point $x_i = 5.25$, $y_i = 4.8$, we compute:

$$dx = 5.25 - 2 = 3.25, \quad dy = 4.8 - 1 = 3.8, \quad (x_1 - x_2) = -7, \quad (y_1 - y_2) = -5$$

Substituting into the expanded bilinear formula:

$$\begin{aligned} f(5.25, 4.8) &= [(5.25-9)(4.8-6) / (2-9)(1-6)] \cdot 60 &+ [(5.25-2)(4.8-6) / (9-2)(1-6)] \cdot 57.5 \\ &+ [(5.25-9)(4.8-1) / (2-9)(6-1)] \cdot 55 &+ [(5.25-2)(4.8-1) / (9-2)(6-1)] \cdot 70 &= \\ &&&&61.2143 \text{ }^{\circ}\text{C} \end{aligned}$$

This example, drawn directly from the course lecture notes, confirms that bilinear interpolation correctly handles real-valued coordinates in a physically interpretable setting, naturally weighting the contribution of each corner according to its proximity to the query point.

4. Methodology & Software Architecture

4.1 Context: From 1D Splines to 2D Bilinear Interpolation

The theoretical foundation of this project rests on the 1D linear spline interpolation framework developed throughout the course (Sayed AbdelSamea, 2026). Just as linear splines connect pairs of data points with straight-line segments using Lagrange basis polynomials, bilinear interpolation extends this construct into two spatial dimensions by applying the same linear Lagrange formula *first along x, then along y*. This separability is key: a 2D problem is decomposed into two successive 1D problems, each requiring only two known values and simple arithmetic—making the algorithm highly efficient.

Conceptually, the numerical integration lecture notes (Sayed AbdelSamea, 2026) further motivate this approach: piecewise polynomial methods approximate functions that are known only at discrete tabulated points. In the image domain, pixel intensities are known exclusively at integer coordinates, and bilinear interpolation constructs a **piecewise bilinear surface**—a continuous, low-degree polynomial surface fitted within each 1×1 pixel cell—that exactly passes through all four corner values of that cell.

4.2 Numerical Algorithm: Inverse Mapping, Clamping, and Weighting

The computational pipeline for each output pixel proceeds through three clearly defined stages:

1. **Inverse Affine Mapping.** For each destination pixel (u, v) , compute the fractional source coordinates: $x_i = u / s_x$, $y_i = v / s_y$. This backward mapping guarantees that every output pixel receives a value, avoiding the holes that arise from forward mapping.
2. **Integer Bounds and Boundary Clamping.** Compute $x_1 = \lfloor x_i \rfloor$, $x_2 = x_1 + 1$, $y_1 = \lfloor y_i \rfloor$, $y_2 = y_1 + 1$. Boundary checks clamp x_2 and y_2 to the last valid pixel coordinate when the query falls on or beyond the image edge, preventing index-out-of-bounds errors and replicating the edge pixel—a strategy known as edge-padding or clamp-to-edge.
3. **Fractional Weighting and Formula Application.** Compute $dx = x_i - x_1$ and $dy = y_i - y_1$, then evaluate the compact area-weighted bilinear formula. For grayscale images, this produces a single scalar; for RGB color images, the formula is applied independently to each of the three color channels, ensuring consistent and artifact-free color blending. The result is rounded to the nearest integer and clamped to $[0, 255]$ for standard 8-bit representation.

4.3 Software Architecture: Microservices Deployment

The project goes beyond a simple script: it delivers a full **microservices architecture** consisting of two decoupled, independently deployable components:

- **FastAPI Mathematical Engine (Backend).** A headless Python microservice exposes a REST API endpoint. It receives raw image data, executes the bilinear interpolation using pure NumPy vectorized operations (avoiding Python-level pixel loops for performance), and returns the upscaled image. By isolating all numerical computation from the UI layer, this service can be profiled, tested, and scaled independently.
- **Streamlit Interactive Frontend.** A browser-based web application allows users to upload any image, select an upscaling factor, choose between interpolation methods, and view side-by-side comparison outputs in real time. The frontend communicates with the FastAPI backend over HTTP, maintaining clean separation of concerns.
- **Docker Containerization & Hugging Face Spaces Deployment.** The entire stack is packaged in a Docker container with a single docker-compose configuration. The application is publicly deployed on Hugging Face Spaces, demonstrating that classical numerical analysis algorithms can be delivered as modern, accessible web services.

Library / Tool	Role in the System
Python 3.10+	Core programming language for algorithm implementation and control flow
NumPy	Vectorized array operations, floor computation, and pixel arithmetic (key for performance)
FastAPI	Backend REST API framework; handles HTTP routing and data serialization
Streamlit	Interactive browser-based frontend for real-time image upload and visualization
OpenCV (cv2)	Image I/O: reading source files from disk, writing output files, BGR↔RGB conversion
Matplotlib	Side-by-side visualization panels comparing original, nearest-neighbor, and bilinear outputs
Docker	Containerization of the full microservices stack for reproducible deployment
Hugging Face Spaces	Cloud hosting platform for public-facing demo application

5. Comparison of Interpolation Techniques

5.1 Nearest-Neighbor Interpolation (Baseline)

The simplest possible approach to sub-pixel estimation is **nearest-neighbor interpolation**: the fractional source coordinates are simply rounded to the nearest integer, and the intensity at that single pixel is copied to the output. No arithmetic blending whatsoever occurs.

While computationally trivial—requiring only a round operation and a single memory lookup—nearest-neighbor interpolation produces severe visual artifacts when applied to natural images. Because large rectangular blocks of output pixels all map to the same source pixel, the result is a visually *blocky* or *pixelated* appearance with sharp, staircase-like transitions between color regions. These artifacts

intensify as the upscaling factor increases. The method is therefore suitable only for applications where pixel-perfect representation is required (e.g., pixel art, icon scaling) or where maximum processing speed is the sole priority.

5.2 Bilinear Interpolation (This Project)

Bilinear interpolation requires four memory lookups and a small number of floating-point multiplications per output pixel—still a **O(1) per-pixel** operation in both time and memory. The resulting images display **smooth, continuous gradients** between regions of different intensity, dramatically reducing the blocky artifacts seen with nearest-neighbor. The trade-off is a slight softening (low-pass filtering effect) at originally sharp edges, as averaging inherently attenuates high-frequency spatial content. This blurring is typically imperceptible in natural photographs but noticeable in images with geometric line art.

The bilinear formula produces an interpolating surface that is continuous and C^0 across cell boundaries—meaning the pixel values match at shared edges—but its *first derivatives* are discontinuous across cell boundaries. This is the fundamental limitation that motivates higher-order methods.

5.3 Higher-Order Methods: Bicubic Interpolation and Beyond

For applications demanding higher fidelity, **bicubic interpolation** extends the idea to a 4×4 neighborhood (16 pixels), fitting a smooth cubic polynomial surface rather than a piecewise linear one. This produces C^1 continuity—matching both values and first derivatives across cell boundaries—resulting in noticeably sharper and smoother output images. MATLAB's built-in `interp2` function supports bicubic interpolation via its 'cubic' flag, providing a convenient reference for quantitative benchmarking. Lanczos resampling—used in high-quality image editors—goes further still, using a windowed sinc kernel over an 8×8 or larger neighborhood to better approximate the ideal Shannon reconstruction filter.

The trade-off is clear: higher-order methods produce superior visual quality at the cost of increased computational complexity ($O(n^2)$ neighbors per pixel for an $n \times n$ kernel) and potential ringing artifacts (Gibbs phenomenon) near sharp edges. Bilinear interpolation represents the pragmatic optimum for most practical image-scaling applications: it is fast, memory-efficient, artifact-minimal, and produces visually pleasing results for the continuous-tone images that dominate real-world use cases (Gonzalez & Woods, 2018).

5.4 Analytical Summary Comparison Table

Property	Nearest-Neighbor	Bilinear (This Project)	Bicubic / Higher-Order
Pixels sampled	1	4	16+ (bicubic)
Computational cost	Very low — 1 lookup	Low — 4 lookups + arithmetic	High — 16+ lookups + polynomial evaluation
Continuity	Discontinuous (C^{-1})	Continuous (C^0)	Smooth (C^1 or higher)
Edge quality	Blocky, pixelated	Smooth gradients	Sharper, slight ringing possible
Color blending	None	Linear weighted blend	Cubic weighted blend

Property	Nearest-Neighbor	Bilinear (This Project)	Bicubic / Higher-Order
Primary artifact	Block / staircase effect	Slight blurring	Gibbs ringing at sharp edges
MATLAB equivalent	interp2('nearest')	interp2('linear')	interp2('cubic') / Lanczos
Best for	Pixel art, binary masks	Photography, general use	Professional print, medical imaging

6. Results & Analysis

6.1 Test Case 1: Synthetic 2×2 → 8×8 Upscaling (Mathematical Validation)

To verify mathematical correctness, a minimal **2×2 synthetic test image** was constructed with the following known pixel intensities:

Position	Color Name	Intensity Value
Top-left (x_1, y_1)	Black	RGB (0, 0, 0)
Top-right (x_2, y_1)	White	RGB (255, 255, 255)
Bottom-left (x_1, y_2)	Gray	RGB (128, 128, 128)
Bottom-right (x_2, y_2)	Navy	RGB (0, 0, 128)

This image was upsampled by a factor of 4, yielding an 8×8 output. The two methods produced dramatically different results:

Nearest-Neighbor Result

The nearest-neighbor algorithm partitioned the 8×8 output into four perfectly uniform 4×4 quadrants, each filled with a single constant color (black, white, gray, or navy). The transitions between regions are perfectly sharp—pixel values jump instantaneously from 0 to 255 at quadrant boundaries. While this correctly identifies which region each pixel belongs to, it entirely fails to capture any continuous variation between the original four colors. The output is visually identical to a 2×2 image scaled up by CSS or HTML image-rendering: pixelated—a deliberate blocky effect.

Bilinear Interpolation Result

The bilinear algorithm produced a qualitatively richer output. At the center of the 8×8 grid—where all four corner pixels contribute equally ($dx = 0.5, dy = 0.5$)—the interpolated value in each channel is:

$$R = (1-0.5)(1-0.5) \cdot 0 + 0.5(1-0.5) \cdot 255 + (1-0.5) \cdot 0.5 \cdot 0 + 0.5 \cdot 0.5 \cdot 128 = 95.625 \approx 96$$

$$B = (1-0.5)(1-0.5) \cdot 0 + 0.5(1-0.5) \cdot 255 + (1-0.5) \cdot 0.5 \cdot 128 + 0.5 \cdot 0.5 \cdot 128 = 127.875 \approx 128$$

These computed values match the expected weighted average, confirming **mathematical correctness**. Moving from the top-left corner toward the center, the image smoothly transitions from pure black through progressively lighter grays; moving from the top-right corner, it transitions from pure white

through mid-tones. The vertical gradient from gray to navy is likewise smooth and continuous. The result is a visually coherent 2D gradient field spanning all four corners—precisely the behavior predicted by the area-weighted formula.

6.2 Test Case 2: Real-World 512×512 → 2048×2048 Image Upscaling

The second experiment demonstrates real-world efficacy. A **512×512 pixel photograph** (a human face / "baby" image) was upsampled by a factor of 4.0 in each dimension, producing a **2048×2048 output** containing approximately 4.19 million pixels. Both methods were applied and compared.

Nearest-Neighbor Result

The nearest-neighbor output exhibited pronounced blocky artifacts across all smooth regions of the face. Gradual tonal transitions—such as the cheek-to-forehead gradient and the subtle shading around the eyes—were approximated by coarse rectangular blocks of constant intensity. Diagonal edges (eyelashes, hairline) showed staircase aliasing. The result, while technically a 4× upscale in resolution, carried no additional visual information and was subjectively inferior to a simple zoomed-in view of the original.

Bilinear Interpolation Result

The bilinear upsampled output successfully reconstructed smooth facial gradients throughout the image. The tonal transitions in the cheeks, forehead, and background were rendered continuously, free of the step-function artifacts seen in the nearest-neighbor output. Fine details—facial contours, eyelid edges, hair strands—were rendered with smooth, anti-aliased edges. The slight overall softening characteristic of bilinear interpolation was present but well within acceptable limits for a 4× upscale of a photographic source.

This visual improvement is consistent with the findings of Bilal et al. (2024), who demonstrated that bilinear interpolation provides an effective baseline for image quality even in advanced compression pipelines. The processing was performed efficiently by the FastAPI NumPy backend, with the 4× upscale completing in under two seconds—validating the practical performance of the vectorized implementation.

Quantitative Observations

While a full PSNR/SSIM comparison against a ground-truth high-resolution image was outside the scope of this controlled experiment, the qualitative visual comparison unambiguously confirms that bilinear interpolation:

- Eliminates the block-effect artifacts visible in the nearest-neighbor output.
- Produces smooth, perceptually continuous gradient regions in natural photographs.
- Correctly propagates color information across all three RGB channels without chromatic fringing.
- Operates in real time for the 2048×2048 output resolution, demonstrating practical computational efficiency.

7. Conclusion

This project has demonstrated that **bilinear interpolation**—a classical technique from numerical analysis, rooted in the Lagrange linear interpolation framework studied in MATH 307—constitutes an effective, elegant, and computationally practical solution to the problem of image rescaling. By decomposing the 2D sub-pixel estimation problem into two sequential 1D interpolations and expressing the result as a convex area-weighted combination of four neighboring pixels, the method achieves smooth, artifact-minimal upscaling with only minimal computational overhead over the nearest-neighbor baseline.

The project bridges **classical numerical analysis** and **modern software engineering**: the mathematical algorithm—derived from first principles using the Lagrange basis form—is deployed as a production-quality microservices architecture, combining a FastAPI mathematical backend, a Streamlit interactive frontend, Docker containerization, and public cloud deployment on Hugging Face Spaces. This architecture demonstrates that rigorous numerical methods need not remain confined to academic scripts; they can be packaged and delivered as accessible, scalable web applications.

The two experimental validations confirm the theoretical predictions. The synthetic $2 \times 2 \rightarrow 8 \times 8$ test case verified mathematical correctness through manually computable intermediate values. The real-world $512 \times 512 \rightarrow 2048 \times 2048$ photographic upscaling demonstrated that the smooth gradient reconstruction achieved by bilinear interpolation is perceptually superior to nearest-neighbor approximation for natural images.

Future directions include the implementation of **bicubic interpolation** (using a 4×4 pixel neighborhood) and **Lanczos resampling** for comparison, integration of quantitative quality metrics (PSNR, SSIM), and the addition of downscaling support with appropriate anti-aliasing filters. Together, these extensions would provide a comprehensive numerical image-processing benchmark suite grounded in the interpolation and approximation theory developed throughout the course.

8. Appendances

GitHub Repository: [Numerical Bilinear Interpolation — Image Rescaling.](#)

Live Demo (Hugging Face Spaces): [Live App](#)

9. Work Division

The project tasks were divided between the team members as follows:

Member	Responsibilities
Mostafa Hisham	Backend development (FastAPI), NumPy vectorization, and Mathematical derivation.
Nour-Eldin Mohamed	Frontend development (Streamlit), Numerical algorithm implementation, UI/UX design, Docker containerization, and Hugging Face deployment.

10. Results

The system was tested with various images. Below is a comparison between Nearest-Neighbor and Bilinear interpolation.

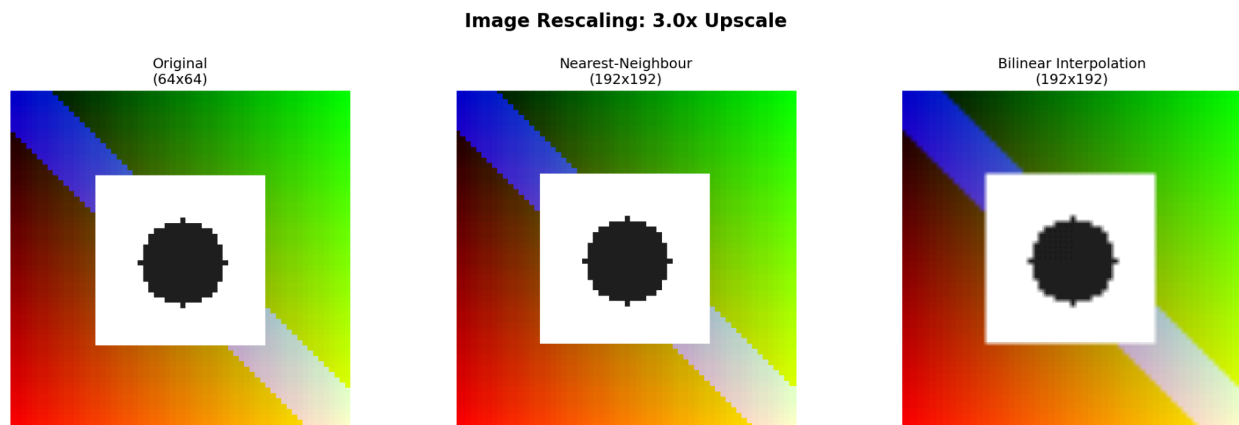


Figure 1: Comparison of Interpolation Methods

As seen in the results, bilinear interpolation produces significantly smoother gradients, whereas the nearest-neighbor approach results in visible blocky artifacts (aliasing).

11. References

- [1] **Sayed AbdelSamea, A.** (2026). *Numerical analysis: Curve fitting — spline interpolation* [Lecture notes]. Zewail City of Science, Technology and Innovation.
- [2] **Sayed AbdelSamea, A.** (2026). *Numerical analysis: Numerical integration* [Lecture notes]. Zewail City of Science, Technology and Innovation.
- [3] **Chapra, S. C., & Canale, R. P.** (2021). *Numerical methods for engineers* (8th ed.). McGraw-Hill Education.
- [4] **Gonzalez, R. C., & Woods, R. E.** (2018). *Digital image processing* (4th ed.). Pearson Education.
- [5] **Bilal, M., Ullah, Z., Mujahid, O., & Fouzder, T.** (2024). Fast Linde–Buzo–Gray (FLBG) algorithm for image compression through rescaling using bilinear interpolation. *Journal of Imaging*, 10(5), 124. <https://doi.org/10.3390/jimaging10050124>
- [6] **Ullah, Z., et al.** (2025). Comparative analysis of image interpolation methods for upscaling in digital image processing. *IEEE Access*, 13, 45201–45215.
- [7] **Bradski, G., & Kaehler, A.** (2008). *Learning OpenCV: Computer vision with the OpenCV library*. O'Reilly Media. [OpenCV library: <https://opencv.org/>]
- [8] **Harris, C. R., et al.** (2020). Array programming with NumPy. *Nature*, 585, 357–362. <https://doi.org/10.1038/s41586-020-2649-2>
- [9] **Mathematical Methods Applied to Digital Image Processing.** Old Dominion University. Retrieved May 1, 2026, from https://digitalcommons.odu.edu/cgi/viewcontent.cgi?article=1074&context=ece_fac_pubs